

Лабораторная работа: Поиск элемента

1.0

Generated by Doxygen 1.17.0

1	Directory Hierarchy	1
1.1	Directories	1
2	Class Index	3
2.1	Class List	3
3	File Index	5
3.1	File List	5
4	Directory Documentation	7
4.1	my_env/lib/python3.14/site-packages/numpy/_core Directory Reference	7
4.2	my_env/lib/python3.14/site-packages/numpy/_core/tests/data Directory Reference	7
4.3	my_env/lib Directory Reference	7
4.4	my_env Directory Reference	7
4.5	my_env/lib/python3.14/site-packages/numpy Directory Reference	7
4.6	my_env/lib/python3.14 Directory Reference	8
4.7	my_env/lib/python3.14/site-packages Directory Reference	8
4.8	my_env/lib/python3.14/site-packages/numpy/_core/tests Directory Reference	8
5	Class Documentation	9
5.1	FlightRecord Struct Reference	9
5.1.1	Detailed Description	9
5.1.2	Member Data Documentation	9
5.1.2.1	airline	9
5.1.2.2	arrivalDate	10
5.1.2.3	arrivalTime	10
5.1.2.4	flightNumber	10
5.1.2.5	passengers	10
5.2	ufunc Struct Reference	10
5.2.1	Member Data Documentation	10
5.2.1.1	f32func	10
5.2.1.2	f32ulp	11
5.2.1.3	f64func	11
5.2.1.4	f64ulp	11
5.2.1.5	name	11
6	File Documentation	13
6.1	BSTree.cpp File Reference	13
6.1.1	Detailed Description	13
6.2	Flight.cpp File Reference	13
6.2.1	Detailed Description	13
6.2.2	Function Documentation	14
6.2.2.1	operator<<()	14
6.2.2.2	operator>>()	14
6.3	generate_data.cpp File Reference	14

6.3.1 Detailed Description	15
6.3.2 Использование	15
6.3.3 Выходные данные	15
6.3.4 Function Documentation	15
6.3.4.1 gen()	15
6.3.4.2 main()	16
6.3.4.3 randomAirline()	16
6.3.4.4 randomDate()	16
6.3.4.5 randomFlightNumber()	16
6.3.4.6 randomPassengers()	17
6.3.4.7 randomTime()	17
6.3.4.8 saveToCSV()	17
6.3.5 Variable Documentation	17
6.3.5.1 AIRLINES	17
6.3.5.2 PREFIXES	18
6.3.5.3 RANDOM_SEED	18
6.3.5.4 UNIQUE_FLIGHT_NUMBERS	18
6.4 HashTable.cpp File Reference	18
6.4.1 Detailed Description	18
6.5 main.cpp File Reference	18
6.5.1 Detailed Description	19
6.5.2 Function Documentation	19
6.5.2.1 getUniqueKeys()	19
6.5.2.2 linearSearch()	20
6.5.2.3 loadFromCSV()	20
6.5.2.4 main()	20
6.6 my_env/lib/python3.14/site-packages/numpy/_core/tests/data/generate_umath_↵ validation_data.cpp File Reference	20
6.6.1 Macro Definition Documentation	21
6.6.1.1 INF	21
6.6.1.2 MAXFLT	21
6.6.1.3 MINDEN	21
6.6.1.4 MINFLT	21
6.6.1.5 qNAN	21
6.6.1.6 sNAN	22
6.6.2 Function Documentation	22
6.6.2.1 append_random_array()	22
6.6.2.2 computeTrueVal()	22
6.6.2.3 generate_input_vector()	22
6.6.2.4 main()	22
6.6.2.5 RandomFloat()	22
6.7 RBTree.cpp File Reference	22
6.7.1 Detailed Description	22

Предметный указатель	23
----------------------	----

Глава 1

Directory Hierarchy

1.1 Directories

_core	7
tests	8
data	7
generate_umath_validation_data.cpp	20
data	7
generate_umath_validation_data.cpp	20
lib	7
python3.14	8
site-packages	8
numpy	7
_core	7
tests	8
data	7
generate_umath_validation_data.cpp	20
my_env	7
lib	7
python3.14	8
site-packages	8
numpy	7
_core	7
tests	8
data	7
generate_umath_validation_data.cpp	20
numpy	7
_core	7
tests	8
data	7
generate_umath_validation_data.cpp	20
python3.14	8
site-packages	8
numpy	7
_core	7
tests	8
data	7
generate_umath_validation_data.cpp	20

site-packages	8
numpy	7
_core	7
tests	8
data	7
generate_umath_validation_data.cpp	20
tests	8
data	7
generate_umath_validation_data.cpp	20

Глава 2

Class Index

2.1 Class List

Here are the classes, structs, unions and interfaces with brief descriptions:

FlightRecord	Вспомогательная структура для временного хранения записи	9
ufunc		10

Глава 3

File Index

3.1 File List

Here is a list of all files with brief descriptions:

BSTree.cpp	
Реализация методов бинарного дерева поиска	13
Flight.cpp	
Реализация ввода/вывода Flight	13
generate_data.cpp	
Генератор тестовых данных для лабораторной работы №2 (поиск данных)	14
HashTable.cpp	
Реализация хэш-таблицы	18
main.cpp	
Программа для сравнения времени поиска в различных структурах данных . . .	18
RBTree.cpp	
Реализация методов красно-чёрного дерева	22
my_env/lib/python3.14/site-packages/numpy/_core/tests/data/ generate_umath_validation_data.cpp	
20	

Глава 4

Directory Documentation

4.1 my_env/lib/python3.14/site-packages/numpy/_core Directory Reference

Directories

- directory [tests](#)

4.2 my_env/lib/python3.14/site-packages/numpy/_core/tests/data Directory Reference

Files

- file [generate_umath_validation_data.cpp](#)

4.3 my_env/lib Directory Reference

Directories

- directory [python3.14](#)

4.4 my_env Directory Reference

Directories

- directory [lib](#)

4.5 my_env/lib/python3.14/site-packages/numpy Directory Reference

Directories

- directory [_core](#)

4.6 my_env/lib/python3.14 Directory Reference

Directories

- directory [site-packages](#)

4.7 my_env/lib/python3.14/site-packages Directory Reference

Directories

- directory [numpy](#)

4.8 my_env/lib/python3.14/site-packages/numpy/_core/tests Directory Reference

Directories

- directory [data](#)

Глава 5

Class Documentation

5.1 FlightRecord Struct Reference

Вспомогательная структура для временного хранения записи.

Public Attributes

- `std::string` [flightNumber](#)
Номер рейса (ключ).
- `std::string` [airline](#)
Авиакомпания
- `std::string` [arrivalDate](#)
Дата прилёта
- `std::string` [arrivalTime](#)
Время прилёта
- `int` [passengers](#)
Число пассажиров

5.1.1 Detailed Description

Вспомогательная структура для временного хранения записи.

Не является частью основной программы, используется только при генерации CSV-файлов.

5.1.2 Member Data Documentation

5.1.2.1 airline

`std::string` `FlightRecord::airline`

Авиакомпания

5.1.2.2 arrivalDate

`std::string FlightRecord::arrivalDate`

Дата прилёта

5.1.2.3 arrivalTime

`std::string FlightRecord::arrivalTime`

Время прилёта

5.1.2.4 flightNumber

`std::string FlightRecord::flightNumber`

Номер рейса (ключ).

5.1.2.5 passengers

`int FlightRecord::passengers`

Число пассажиров

The documentation for this struct was generated from the following file:

- [generate_data.cpp](#)

5.2 ufunc Struct Reference

Public Attributes

- `std::string` [name](#)
- `double(* f32func )(double)`
- `long double(* f64func )(long double)`
- `float` [f32ulp](#)
- `float` [f64ulp](#)

5.2.1 Member Data Documentation

5.2.1.1 f32func

`double(* ufunc::f32func) (double)`

5.2.1.2 f32ulp

float ufunc::f32ulp

5.2.1.3 f64func

long double(* ufunc::f64func) (long double)

5.2.1.4 f64ulp

float ufunc::f64ulp

5.2.1.5 name

std::string ufunc::name

The documentation for this struct was generated from the following file:

- `my_env/lib/python3.14/site-packages/numpy/_core/tests/data/generate_umath_validation_data.cpp`

Глава 6

File Documentation

6.1 BSTree.cpp File Reference

Реализация методов бинарного дерева поиска.

```
#include "BSTree.hpp"
```

6.1.1 Detailed Description

Реализация методов бинарного дерева поиска.

6.2 Flight.cpp File Reference

Реализация ввода/вывода Flight.

```
#include "Flight.hpp"  
#include <sstream>
```

Functions

- istream & [operator>>](#) (istream &is, Flight &f)
- ostream & [operator<<](#) (ostream &os, const Flight &f)

6.2.1 Detailed Description

Реализация ввода/вывода Flight.

6.2.2 Function Documentation

6.2.2.1 operator<<()

```
ostream & operator<< (  
    ostream & os,  
    const Flight & f)
```

6.2.2.2 operator>>()

```
istream & operator>> (  
    istream & is,  
    Flight & f)
```

6.3 generate_data.cpp File Reference

Генератор тестовых данных для лабораторной работы №2 (поиск данных).

```
#include <iostream>  
#include <fstream>  
#include <string>  
#include <vector>  
#include <random>  
#include <iomanip>  
#include <sstream>  
#include <sys/stat.h>
```

Classes

- struct [FlightRecord](#)
Вспомогательная структура для временного хранения записи.

Functions

- std::mt19937 [gen](#) ([RANDOM_SEED](#))
Глобальный генератор случайных чисел (Mersenne Twister).
- std::string [randomDate](#) ()
Генерация случайной даты прилёта.
- std::string [randomTime](#) ()
Генерация случайного времени прилёта.
- std::string [randomAirline](#) ()
Генерация случайного названия авиакомпании.
- int [randomPassengers](#) ()
Генерация случайного числа пассажиров.
- std::string [randomFlightNumber](#) ()
Генерация номера рейса из ограниченного множества.
- void [saveToCSV](#) (const std::vector< [FlightRecord](#) > &flights, const std::string &filename)
Сохранение вектора записей в CSV-файл.
- int [main](#) ()
Главная функция генератора данных.

Variables

- const int `UNIQUE_FLIGHT_NUMBERS` = 2000
Количество уникальных номеров рейсов.
- const unsigned int `RANDOM_SEED` = 42
Фиксированный seed для генератора (воспроизводимость).
- const std::vector< std::string > `AIRLINES`
Список авиакомпаний для случайного выбора.
- const std::vector< std::string > `PREFIXES` = {"SU", "S7", "FV", "DP", "U6", "UT", "N4", "WZ"}
Префиксы для номеров рейсов.

6.3.1 Detailed Description

Генератор тестовых данных для лабораторной работы №2 (поиск данных).

Создаёт CSV-файлы с информацией об авиарейсах. Номера рейсов (ключи поиска) генерируются из ограниченного набора, что обеспечивает многократные повторения для тестирования поиска всех вхождений.

6.3.2 Использование

```
./generate_data
```

6.3.3 Выходные данные

Файлы сохраняются в папку data/ с именами flights_{size}.csv, где size – количество записей.

See also

Flight (класс из основной программы)
main_benchmark.cpp

6.3.4 Function Documentation

6.3.4.1 gen()

```
std::mt19937 gen (  
    RANDOM_SEED )
```

Глобальный генератор случайных чисел (Mersenne Twister).

Инициализируется фиксированным seed для воспроизводимости.

6.3.4.2 main()

```
int main ()
```

Главная функция генератора данных.

Returns

0 при успешном завершении.

Последовательно генерирует файлы для следующих размеров: 100, 500, 1000, 5000, 10000, 20000, 30000, 50000, 75000, 100000, 200000, 300000, 500000, 750000, 1000000.

Postcondition

В папке data/ создаются файлы flights_{size}.csv.

6.3.4.3 randomAirline()

```
std::string randomAirline ()
```

Генерация случайного названия авиакомпании.

Returns

Одна из строк в списке AIRLINES.

6.3.4.4 randomDate()

```
std::string randomDate ()
```

Генерация случайной даты прилёта.

Returns

Строка в формате YYYY-MM-DD.

Диапазон: 2024-01-01 до 2025-12-28.

6.3.4.5 randomFlightNumber()

```
std::string randomFlightNumber ()
```

Генерация номера рейса из ограниченного множества.

Returns

Строка вида "SU1234", где число от 1 до UNIQUE_FLIGHT_NUMBERS.

Номер рейса формируется как случайный префикс + число. Такая схема гарантирует, что количество различных ключей не превышает `UNIQUE_FLIGHT_NUMBERS * PREFIXES.size()`, что создаёт множество дубликатов в больших массивах.

6.3.4.6 randomPassengers()

```
int randomPassengers ()
```

Генерация случайного числа пассажиров.

Returns

Число от 0 до 300.

6.3.4.7 randomTime()

```
std::string randomTime ()
```

Генерация случайного времени прилёта.

Returns

Строка в формате HH:MM:SS.

Диапазон: 00:00:00 до 23:59:59.

6.3.4.8 saveToCSV()

```
void saveToCSV (  
    const std::vector< FlightRecord > & flights,  
    const std::string & filename)
```

Сохранение вектора записей в CSV-файл.

Parameters

flights	Вектор структур FlightRecord .
filename	Имя выходного файла.

Заголовок файла соответствует ожиданиям класса `Flight` из основной программы.

6.3.5 Variable Documentation

6.3.5.1 AIRLINES

```
const std::vector<std::string> AIRLINES
```

Initial value:

```
= {  
    "Aeroflot", "S7 Airlines", "Rossiya", "Pobeda",  
    "Ural Airlines", "Utair", "Nordwind", "Red Wings"  
}
```

Список авиакомпаний для случайного выбора.

6.3.5.2 PREFIXES

```
const std::vector<std::string> PREFIXES = {"SU", "S7", "FV", "DP", "U6", "UT", "N4", "WZ"}
```

Префиксы для номеров рейсов.

6.3.5.3 RANDOM_SEED

```
const unsigned int RANDOM_SEED = 42
```

Фиксированный seed для генератора (воспроизводимость).

6.3.5.4 UNIQUE_FLIGHT_NUMBERS

```
const int UNIQUE_FLIGHT_NUMBERS = 2000
```

Количество уникальных номеров рейсов.

Чем меньше это число, тем больше дубликатов в массиве. При размере N каждый ключ в среднем встретится $N / \text{UNIQUE_COUNT}$ раз.

6.4 HashTable.cpp File Reference

Реализация хэш-таблицы.

```
#include "HashTable.hpp"
```

6.4.1 Detailed Description

Реализация хэш-таблицы.

6.5 main.cpp File Reference

Программа для сравнения времени поиска в различных структурах данных.

```
#include <iostream>
#include <fstream>
#include <vector>
#include <string>
#include <chrono>
#include <map>
#include <algorithm>
#include <set>
#include "Flight.hpp"
#include "BSTree.hpp"
#include "RBTree.hpp"
#include "HashTable.hpp"
```


Functions

- `vector< Flight > loadFromCSV (const string &filename)`
Загрузка данных из CSV-файла.
- `vector< Flight > linearSearch (const vector< Flight > &data, const string &key)`
Линейный поиск всех вхождений ключа в массиве.
- `vector< string > getUniqueKeys (const vector< Flight > &data, size_t count)`
Получение первых N уникальных ключей из массива.
- `int main ()`

6.5.1 Detailed Description

Программа для сравнения времени поиска в различных структурах данных.

Загружает данные об авиарейсах из CSV-файлов разного размера, строит структуры поиска (BST, красно-чёрное дерево, хэш-таблица, `std::multimap`) и замеряет время выполнения 10 поисковых запросов. Результаты сохраняются в CSV-файл для последующего построения графиков.

See also

Flight, BSTree, RBTree, HashTable

6.5.2 Function Documentation

6.5.2.1 getUniqueKeys()

```
vector< string > getUniqueKeys (  
    const vector< Flight > & data,  
    size_t count)
```

Получение первых N уникальных ключей из массива.

Parameters

data	Вектор объектов.
count	Желаемое количество уникальных ключей.

Returns

Вектор уникальных ключей (не более count).

6.5.2.2 linearSearch()

```
vector< Flight > linearSearch (
    const vector< Flight > & data,
    const string & key)
```

Линейный поиск всех вхождений ключа в массиве.

Parameters

data	Вектор объектов.
key	Искомый ключ (номер рейса).

Returns

Вектор найденных объектов.

6.5.2.3 loadFromCSV()

```
vector< Flight > loadFromCSV (
    const string & filename)
```

Загрузка данных из CSV-файла.

Parameters

filename	Имя файла.
----------	------------

Returns

Вектор объектов Flight.

6.5.2.4 main()

```
int main ()
```

6.6 my_env/lib/python3.14/site-packages/numpy/_core/tests/data/↵ generate_umath_validation_data.cpp File Reference

```
#include <algorithm>
#include <fstream>
#include <iostream>
#include <math.h>
#include <random>
#include <cstdio>
#include <ctime>
#include <vector>
```

Classes

- struct [ufunc](#)

Macros

- `#define` [MINDEN](#) `std::numeric_limits<T>::denorm_min()`
- `#define` [MINFLT](#) `std::numeric_limits<T>::min()`
- `#define` [MAXFLT](#) `std::numeric_limits<T>::max()`
- `#define` [INF](#) `std::numeric_limits<T>::infinity()`
- `#define` [qNaN](#) `std::numeric_limits<T>::quiet_NaN()`
- `#define` [sNaN](#) `std::numeric_limits<T>::signaling_NaN()`

Functions

- `template<typename T>`
`T` [RandomFloat](#) (`T a`, `T b`)
- `template<typename T>`
`void` [append_random_array](#) (`std::vector< T > &arr`, `T min`, `T max`, `size_t N`)
- `template<typename T1, typename T2>`
`std::vector< T1 >` [computeTrueVal](#) (`const std::vector< T1 > &in`, `T2(*mathfunc)(T2)`)
- `template<typename T>`
`std::vector< T >` [generate_input_vector](#) (`std::string func`)
- `int` [main](#) ()

6.6.1 Macro Definition Documentation

6.6.1.1 INF

```
#define INF std::numeric_limits<T>::infinity()
```

6.6.1.2 MAXFLT

```
#define MAXFLT std::numeric_limits<T>::max()
```

6.6.1.3 MINDEN

```
#define MINDEN std::numeric_limits<T>::denorm_min()
```

6.6.1.4 MINFLT

```
#define MINFLT std::numeric_limits<T>::min()
```

6.6.1.5 qNaN

```
#define qNaN std::numeric_limits<T>::quiet_NaN()
```

6.6.1.6 sNaN

```
#define sNaN std::numeric_limits<T>::signaling_NaN()
```

6.6.2 Function Documentation

6.6.2.1 append_random_array()

```
template<typename T>
void append_random_array (
    std::vector< T > & arr,
    T min,
    T max,
    size_t N)
```

6.6.2.2 computeTrueVal()

```
template<typename T1, typename T2>
std::vector< T1 > computeTrueVal (
    const std::vector< T1 > & in,
    T2(* mathfunc )(T2))
```

6.6.2.3 generate_input_vector()

```
template<typename T>
std::vector< T > generate_input_vector (
    std::string func)
```

6.6.2.4 main()

```
int main ()
```

6.6.2.5 RandomFloat()

```
template<typename T>
T RandomFloat (
    T a,
    T b)
```

6.7 RBTREE.cpp File Reference

Реализация методов красно-чёрного дерева.

```
#include "RBTREE.hpp"
```

6.7.1 Detailed Description

Реализация методов красно-чёрного дерева.

Предметный указатель

- airline
 - FlightRecord, 9
- AIRLINES
 - generate_data.cpp, 17
- append_random_array
 - generate_umath_validation_data.cpp, 22
- arrivalDate
 - FlightRecord, 9
- arrivalTime
 - FlightRecord, 10
- BSTree.cpp, 13
- computeTrueVal
 - generate_umath_validation_data.cpp, 22
- f32func
 - ufunc, 10
- f32ulp
 - ufunc, 10
- f64func
 - ufunc, 11
- f64ulp
 - ufunc, 11
- Flight.cpp, 13
 - operator<<, 14
 - operator>>, 14
- flightNumber
 - FlightRecord, 10
- FlightRecord, 9
 - airline, 9
 - arrivalDate, 9
 - arrivalTime, 10
 - flightNumber, 10
 - passengers, 10
- gen
 - generate_data.cpp, 15
- generate_data.cpp, 14
 - AIRLINES, 17
 - gen, 15
 - main, 15
 - PREFIXES, 17
 - RANDOM_SEED, 18
 - randomAirline, 16
 - randomDate, 16
 - randomFlightNumber, 16
 - randomPassengers, 16
 - randomTime, 17
 - saveToCSV, 17
 - UNIQUE_FLIGHT_NUMBERS, 18
- generate_input_vector
 - generate_umath_validation_data.cpp, 22
- generate_umath_validation_data.cpp
 - append_random_array, 22
 - computeTrueVal, 22
 - generate_input_vector, 22
 - INF, 21
 - main, 22
 - MAXFLT, 21
 - MINDEN, 21
 - MINFLT, 21
 - qNAN, 21
 - RandomFloat, 22
 - sNAN, 21
- getUniqueKeys
 - main.cpp, 19
- HashTable.cpp, 18
- INF
 - generate_umath_validation_data.cpp, 21
- linearSearch
 - main.cpp, 19
- loadFromCSV
 - main.cpp, 20
- main
 - generate_data.cpp, 15
 - generate_umath_validation_data.cpp, 22
 - main.cpp, 20
- main.cpp, 18
 - getUniqueKeys, 19
 - linearSearch, 19
 - loadFromCSV, 20
 - main, 20
- MAXFLT
 - generate_umath_validation_data.cpp, 21
- MINDEN
 - generate_umath_validation_data.cpp, 21
- MINFLT
 - generate_umath_validation_data.cpp, 21
- my_env Directory Reference, 7
- my_env/lib Directory Reference, 7
- my_env/lib/python3.14 Directory Reference, 8
- my_env/lib/python3.14/site-packages Directory Reference, 8
- my_env/lib/python3.14/site-packages/numpy Directory Reference, 7

my_env/lib/python3.14/site-packages/numpy/_core
 Directory Reference, [7](#)
my_env/lib/python3.14/site-packages/numpy/_core/tests
 Directory Reference, [8](#)
my_env/lib/python3.14/site-packages/numpy/_core/tests/data
 Directory Reference, [7](#)
my_env/lib/python3.14/site-packages/numpy/_core/tests/data/generate_umath_validation_data.cpp,
 [20](#)

name
 ufunc, [11](#)

operator<<
 Flight.cpp, [14](#)
operator>>
 Flight.cpp, [14](#)

passengers
 FlightRecord, [10](#)
PREFIXES
 generate_data.cpp, [17](#)

qNaN
 generate_umath_validation_data.cpp, [21](#)

RANDOM_SEED
 generate_data.cpp, [18](#)
randomAirline
 generate_data.cpp, [16](#)
randomDate
 generate_data.cpp, [16](#)
randomFlightNumber
 generate_data.cpp, [16](#)
RandomFloat
 generate_umath_validation_data.cpp, [22](#)
randomPassengers
 generate_data.cpp, [16](#)
randomTime
 generate_data.cpp, [17](#)
RBTREE.cpp, [22](#)

saveToCSV
 generate_data.cpp, [17](#)
sNaN
 generate_umath_validation_data.cpp, [21](#)

ufunc, [10](#)
 f32func, [10](#)
 f32ulp, [10](#)
 f64func, [11](#)
 f64ulp, [11](#)
 name, [11](#)
UNIQUE_FLIGHT_NUMBERS
 generate_data.cpp, [18](#)